

Analysis of Custom CNN, ResNet50, and VGG16 Models for German Traffic Sign Classification

Timothy Hutzley

Task Description:

The classification task for this assignment involves identifying different German traffic signs. To accomplish this task, the data being used is from the GTSRB (German Traffic Sign Recognition Benchmark) dataset.

Dataset:

The dataset itself contains images along with labels classifying each traffic sign. In total there are 43 distinct classes and the images are all provided in varying sizes.

During pre-processing, all images are resized to 32 x 32 for consistency when loading data for the custom CNN or visualization. For the use of the ResNet and VGG models, the images are resized to 224 x 224 to allow for input and data augmentation takes place. For data augmentation, the images are flipped horizontally at random, rotated at an orientation of 15 degrees at random, and the brightness, color, and saturation values are altered using color jitter. These are all in place to help the models better generalize. Lastly, the data is transformed to a tensor where normalization then takes place. For the CNN model the data is normalized using a mean of 0.5 and a standard deviation of 0.5 while for the ResNet and VGG models, the data is normalized using the

mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225] of ImageNet as that is what the models are pre trained on.

Visualization of Data:



Custom CNN Model:

Layer 1:

- 32 filters with a 3x3 kernel size with padding 1
- Batch normalization to normalize activations, improving training stability and performance
- ReLU activation to introduce non-linearity
- 2x2 max pooling to the spatial dimensions of the feature maps by half

Layer 2:

- 64 filters with a 3x3 kernel size with padding 1
- Batch normalization
- ReLU activation
- 2x2 max pooling

Fully Connected Layers:

- Output of the convolutional layers is flattened into a 1D vector
- Map the flattened vector to 512 neurons
- ReLU activation
- Dropout layer with a probability of 0.3 to prevent overfitting by randomly setting 30% of the neurons to zero during training
- Map 512 neurons to number of output classes (43)

Code Implementation Explanation:

The model is initialized in the same way as in HW5 creating a python class where the `nn.Module` is passed as a parameter. Each of the three previously described layers are separated and created using `nn.Sequential`. This function allows for each forward pass of each layer to be grouped together as one. For layer one, the filters, batch normalization, ReLU activation, and max pooling are all combined into one `nn.Sequential`. Then a forward function is defined and in this function the data is then passed through the first layer as defined. The output of that first layer is then passed to the second layer which then lastly passes its output to the fully connected layers. This maps the 512 neurons to the number of classes which is 43, resulting in the final output which are the logits for each class.

ResNet50 and VGG16 Fine-Tuning:

ResNet50 is a deep convolutional neural network that consists of 50 weight layers and introduces “identity shortcut connection” which allows the model to skip one or more layers which makes it possible to train the model on more layers without performance degradation. Some of the key features of this model’s architecture include the usage of convolutional layers, BatchNorm, ReLU, and identity blocks. The original fully connected layers result in an output of 1000 classes, but the output has been replaced to output 43 classes which are represented in the GTSRB dataset. The weights used were the default pre-trained weights from ImageNet and were then optimized using a scheduler. This helped the model focus on optimizing the fully connected layers while the earlier layers continued to refine the features.

VGG16 is also a deep convolutional neural network that consists of 16 layers with the use of many 3x3 convolutional layers and max-pooling layers with a fully connected classifier at the end. The original output is the same as ResNet with 1000 classes so that was replaced with an output of 43 classes. The classifier head of the model was also restructured to include both ReLU activation and dropout with a probability of 0.3 to better help the model generalize. Along with dropout, label smoothing with a value of 0.1 was also applied and a weight decay of $1e-8$ was applied to the optimizer.

Along with the following changes, both models used Cross Entropy Loss as the loss function and for the optimizer both models used Adam. Adam optimizer was chosen as it involves faster convergence and it adjusts the magnitudes being used to

update parameters making it useful for networks with many layers. A scheduler was also used when training the models which adjusts the learning rate during training to improve not only generalization but help stabilize the training as well. Lastly, as described during the pre-processing phase, all inputted images were resized to 224 x 224, normalized using the ImageNet mean and standard deviation, random flips and rotations were applied, and lastly color jittering affecting the brightness, color, and saturation of the images were also applied to help the models better generalize.

Hyperparameters Selection:

For all 3 of the models the batch size was set to 32 as it performed consistently well. The number of epochs was also set to 10 to allow the models to learn as much as possible without taking too much time.

The learning rate for the custom CNN architecture was set to 0.001 which was a bit higher than the learning rate of 0.0001 used for both ResNet and VGG. After some testing this was found to be the best values for each model as the higher rate for the custom CNN allowed for steady optimization throughout the training period. Using a smaller learning rate for the pre-trained models ensured that the pre-trained weights weren't changed too much during the training process but just enough to better understand the GTSRB dataset.

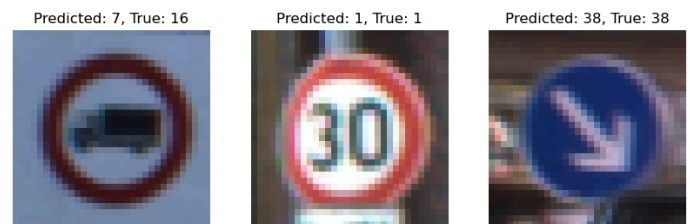
As mentioned before the optimizer Adam was used for all models due to faster convergence and its usefulness when it comes to optimizing networks with many layers. Similarly, as mentioned before, the scheduler was used for all models to help

stabilize training and the model's ability to generalize.

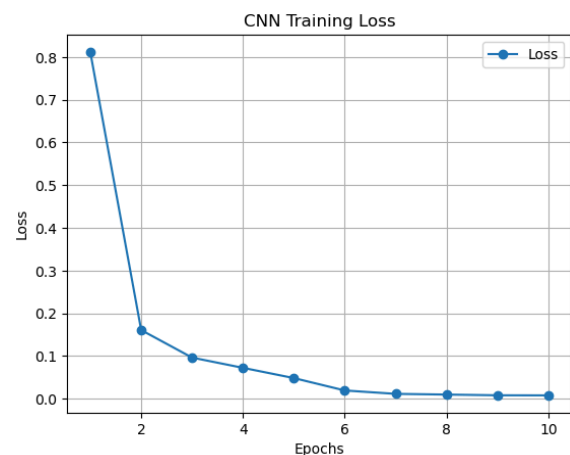
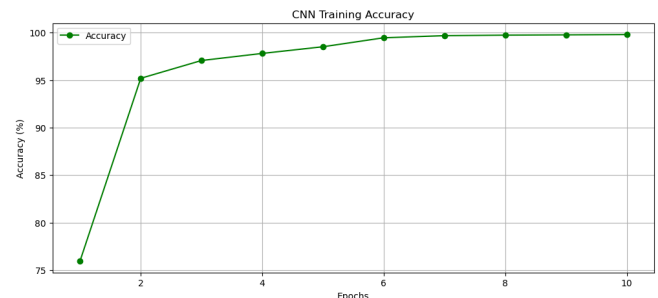
Results and Analysis:

Custom CNN Model had a final test accuracy of 96.96% and a final test loss of 0.1293

Custom CNN Predictions Visualized:

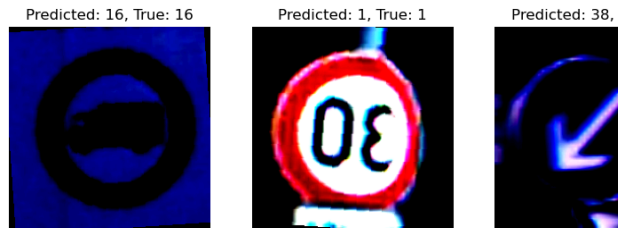


Custom CNN Training Results:

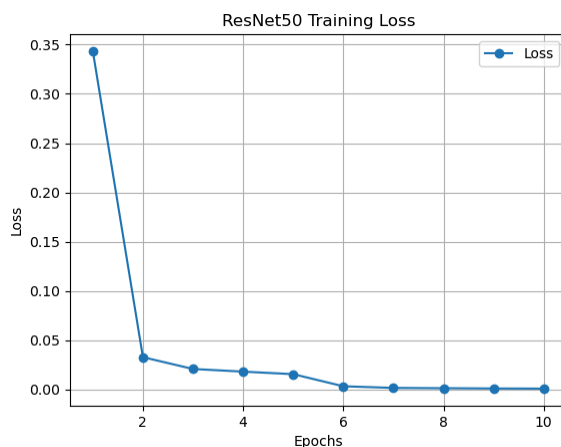
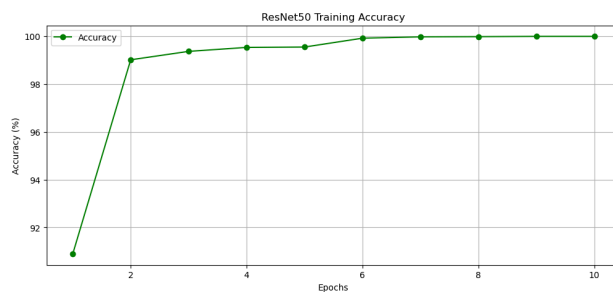


ResNet50 Model had a final test accuracy of 97.22% and a final test loss of 0.1767

ResNet50 Predictions Visualized:

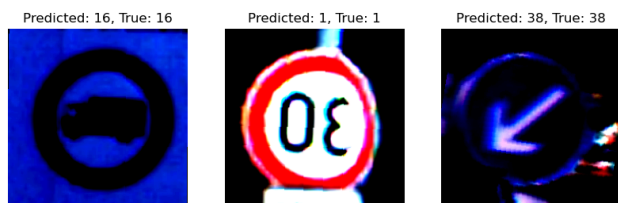


ResNet50 Training Results:

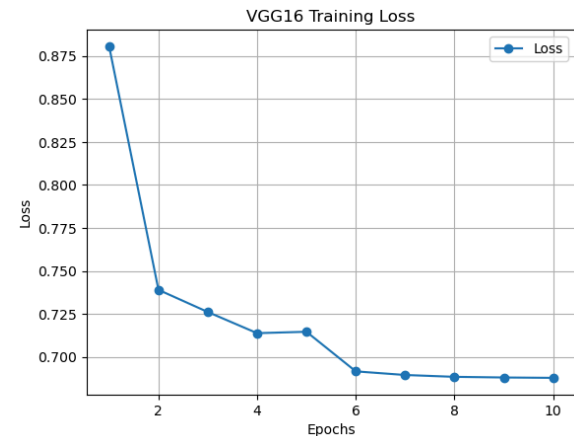
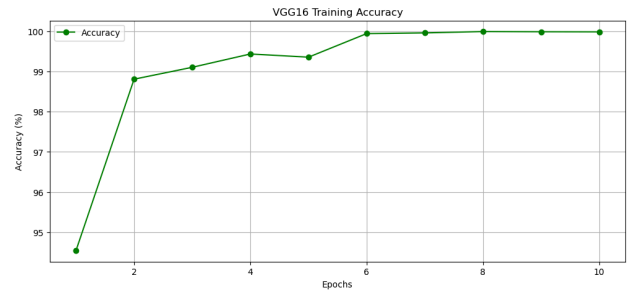


VGG16 Model had a final test accuracy of 97.40% and a final loss of 0.2136

VGG Predictions Visualized:



VGG Training Results:



Conclusion:

Out of all three models, VGG16 performed the best with the highest test accuracy of 97.40%, just barely ahead of ResNet50 with a test accuracy of 97.22%. The custom CNN architecture had the lowest test accuracy, but was still extremely close to the others at 96.96%. In terms of efficiency, the custom CNN model performed nearly as well as both of the pre-trained models at a much faster pace. This model also had the lowest test loss indicating it may be the most stable when it comes to making predictions on unseen data. However, the ResNet50 model had the lowest training loss which suggests the model may have understood the dataset the best. Unlike ResNet50, VGG16 had the highest training loss which could be attributed to label smoothing due to preventing the model from overfitting. Overall, when looking for a computationally efficient model with minimal

complexity, the custom CNN appears to be the prominent choice. When looking for a high accuracy model that performs well at feature extraction and appears to generalize well after fine-tuning, ResNet50 is the best option. Lastly, if there are plenty of resources available and the highest accuracy is the goal, VGG16 is the leading choice due to its high test accuracy.